

LAPORAN PRAKTIKUM
STRUKTUR DATA
“laporan Praktikum Pekan 7”

Disusun Oleh:

Faiz Fikri Satria

2511533026

Dosen Pengampu : Dr. Wahyudi, S.T, M.T.
Asisten Praktikum : Sherly Sukmadira



DAPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga kami dapat menyelesaikan praktikum dan laporan ini dengan baik. Laporan praktikum ini disusun sebagai dokumentasi dari praktikum pekan ke-7 mata kuliah Struktur Data yang membahas implementasi beberapa algoritma sorting dasar, yaitu Insertion Sort, Selection Sort, dan Bubble Sort, serta visualisasinya dalam bentuk Graphical User Interface (GUI) menggunakan Java Swing. Praktikum ini membantu kami memahami cara kerja algoritma pengurutan beserta perbandingan efisiensinya.

Kami menyadari bahwa laporan ini masih memiliki kekurangan baik dari segi teknis maupun penyajian. Oleh karena itu, kritik dan saran yang membangun sangat kami harapkan demi penyempurnaan laporan di masa mendatang.

Padang, 21 Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	1
BAB II PEMBAHASAN	2
2.1 InsertionSort_2511533026	2
2.1.1 Teori	2
2.1.2 Langkah-langkah	2
2.1.3 Contoh Kode	2
2.1.4 Hasil Output	2
2.1.5 Analisis	2
2.2 SelectionSort_2511533026	3
2.2.1 Teori	3
2.2.2 Langkah-langkah	3
2.2.3 Contoh Kode	3
2.2.4 Hasil Output	3
2.2.5 Analisis	3
2.3 BubbleSort_2511533026	4
2.2.1 Teori	4

2.2.2 Langkah-langkah	4
2.2.3 Contoh Kode	4
2.2.4 Hasil Output	4
2.2.5 Analisis	4
2.4 InsertionSortGUI_2511533026.....	5
2.2.1 Teori	5
2.2.2 Langkah-langkah	5
2.2.3 Contoh Kode	5
2.2.4 Hasil	6
2.2.5 Analisis	6
2.6 Tugas Pekan 6	7
2.5.1. Analisis	7
BAB III KESIMPULAN	10
DAFTAR PUSTAKA	11

BAB I

PENDAHULUAN

1.1 Latar Belakang

Algoritma sorting merupakan salah satu topik fundamental dalam ilmu komputer yang digunakan untuk mengurutkan elemen dalam suatu kumpulan data. Praktikum pekan ini memfokuskan pada tiga algoritma sorting sederhana yaitu Insertion Sort, Selection Sort, dan Bubble Sort yang semuanya memiliki kompleksitas waktu $O(n^2)$ pada kasus terburuk. Selain implementasi konvensional, praktikum juga menyajikan visualisasi interaktif menggunakan GUI untuk Insertion Sort.

1.2 Tujuan

1. Memahami konsep dan cara kerja algoritma Insertion Sort, Selection Sort, dan Bubble Sort.
2. Mengimplementasikan ketiga algoritma sorting tersebut dalam bahasa Java.
3. Membandingkan proses pengurutan dari ketiga algoritma.
4. Mengembangkan aplikasi GUI untuk visualisasi langkah per langkah Insertion Sort.
5. Memahami konsep step-by-step visualization pada algoritma sorting.

1.3 Manfaat

1. Mahasiswa memahami perbedaan mekanisme kerja masing-masing algoritma sorting.
2. Meningkatkan kemampuan analisis kompleksitas algoritma dan efisiensi.
3. Memberikan pengalaman dalam pembuatan aplikasi GUI untuk visualisasi algoritma.

BAB II

PEMBAHASAN

2.1 InsertionSort_2511533026

2.1.1 Teori

Insertion Sort bekerja dengan cara membagi array menjadi bagian yang sudah terurut dan belum terurut, kemudian memasukkan elemen satu per satu ke posisi yang tepat pada bagian yang sudah terurut.

2.1.2 Langkah-langkah

1. Mulai dari indeks ke-1 hingga akhir array.
2. Simpan elemen saat ini sebagai key.
3. Geser elemen-elemen yang lebih besar ke kanan.
4. Masukkan key ke posisi yang tepat.

2.1.3 Contoh Kode

```
1 package InsertionSort_2511533026;
2
3 public class InsertionSort_2511533026 {
4     public static void InsertionSort(int arr[]) {
5         int n = arr.length;
6         for (int i = 1; i < n; i++) {
7             int key = arr[i];
8             int j = i - 1;
9             while (j > 0 && arr[j] > key) {
10                 arr[j + 1] = arr[j];
11                 j--;
12             }
13             arr[j + 1] = key;
14         }
15     }
16
17     public static void main(String[] args) {
18         int arr[] = {23, 78, 45, 8, 32, 56, 1};
19         int n = arr.length;
20         InsertionSort(arr);
21         for (int i = 0; i < n; i++) {
22             System.out.print(arr[i] + " ");
23         }
24         System.out.println();
25     }
26 }
```

2.1.4 Hasil Output

```
Console X Install Java 26 Support
<terminated> InsertionSort_2511533026 [Java Application] D:\
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78
```

2.1.5 Analisis

Insertion Sort efisien untuk data yang hampir terurut (best case $O(n)$). Cocok untuk data berukuran kecil hingga sedang.

2.2 SelectionSort_2511533026

2.2.1 Teori

Selection Sort mencari elemen terkecil (atau terbesar) dari bagian yang belum terurut dan menukarnya dengan elemen pertama bagian tersebut.

2.2.2 Langkah-langkah

1. Pada setiap iterasi, cari indeks elemen minimum.
2. Tukar elemen minimum dengan elemen pada posisi iterasi saat ini.

2.2.3 Contoh Kode

```
1 package SelectionSort_2511533026;
2
3 public class SelectionSort_2511533026 {
4     public static void selectionSort(int[] arr) {
5         int n = arr.length;
6         for (int i = 0; i < n - 1; i++) {
7             int minIndex = i;
8             for (int j = i + 1; j < n; j++) {
9                 if (arr[j] < arr[minIndex]) {
10                     minIndex = j;
11                 }
12             }
13             // Swap
14             int temp = arr[i];
15             arr[i] = arr[minIndex];
16             arr[minIndex] = temp;
17         }
18     }
19
20     public static void main(String[] args) {
21         int[] arr = {23, 78, 45, 8, 32, 56, 1};
22         selectionSort(arr);
23         System.out.println("array yang sudah terurut: ");
24         for (int i = 0; i < arr.length; i++) {
25             System.out.print(arr[i] + " ");
26         }
27         System.out.println();
28         selectionSort_2511533026 arr = new SelectionSort_2511533026();
29         System.out.println("array yang terurut: ");
30         for (int i = 0; i < arr.length; i++) {
31             System.out.print(arr[i] + " ");
32         }
33         System.out.println();
34     }
35 }
```

2.2.4 Hasil Output

```
Console X Install Java 26 Support
<terminated> SelectionSort_2511533026 [Java Application] D:\
array yang belum terurut: 23 78 45 8 32 56 1
array yang terurut: 1 8 23 32 45 56 78
```

2.2.5 Analisis

Selection Sort selalu melakukan $O(n^2)$ perbandingan, tidak adaptif terhadap data yang hampir terurut.

2.3 BubbleSort_2511533026

2.3.1 Teori Bubble Sort bekerja dengan cara membandingkan elemen yang berdekatan secara berulang dan menukar posisinya jika urutannya salah.

2.3.2 Langkah-langkah

1. Lakukan perulangan sebanyak $n-1$ kali.
2. Pada setiap pass, bandingkan elemen berdekatan dan tukar jika diperlukan.

2.3.3 Contoh Kode

```
1 // Bubble Sort_2511533026
2
3 public class BubbleSort {
4     public static void main(String[] args) {
5         int arr[] = {23, 78, 45, 8, 32, 56, 1};
6         int n = arr.length;
7
8         for (int i = 0; i < n - 1; i++) {
9             for (int j = 0; j < n - i - 1; j++) {
10                 if (arr[j] > arr[j + 1]) {
11                     // Tukar
12                     int temp = arr[j];
13                     arr[j] = arr[j + 1];
14                     arr[j + 1] = temp;
15                 }
16             }
17         }
18
19         // Menampilkan array setelah diurutkan
20         System.out.println("Array yang sudah terurut:");
21         for (int i = 0; i < n; i++) {
22             System.out.print(arr[i] + " ");
23         }
24         System.out.println();
25     }
26 }
```

2.3.4 Hasil Output

```
Console X Install Java 26 Support
<terminated> BubbleSort_2511533026 [Java Application] D:\ec
Array yang belum terurut:
23 78 45 8 32 56 1

Array yang sudah terurut:
1 8 23 32 45 56 78
```

2.3.5 Analisis

Bubble Sort paling sederhana namun paling lambat di antara ketiganya pada kasus terburuk.


```

114 StringIndexer stepLog_3026 = new StringIndexer();
115 stepLog_3026.append("Langkah 1: "); append(stepCount_3023).append("\n");
116 stepLog_3026.append("Memasukkan ").append(key_3025).append("\n");
117
118 while (j_3026 >= 0 && array_3025[j_3026] > key_3025) {
119     array_3025[j_3026 + 1] = array_3025[j_3026];
120     j_3026--;
121 }
122 array_3025[j_3026 + 1] = key_3025;
123
124 updateLabels_3026();
125 stepLog_3026.append("Hasil: ").append(arrayToString_3026(array_3025)).append("\n\n");
126 stepLog_3026.append(stepLog_3026.toString());
127 i_3026++;
128 stepCount_3024++;
129
130 if (i_3026 >= array_3025.length) {
131     sorting_3025 = false;
132     stepButton_3026.setEnabled(false);
133     stepLog_3026.append("Sorting selesai!\n");
134     JOptionPane.showMessageDialog(this, "Sorting selesai!");
135 }
136
137 private void updateLabels_3026() {
138     for (int k_3026 = 0; k_3026 < array_3025.length; k_3026++) {
139         labelArray_3025[k_3026].setText(String.valueOf(array_3025[k_3026]));
140     }
141 }

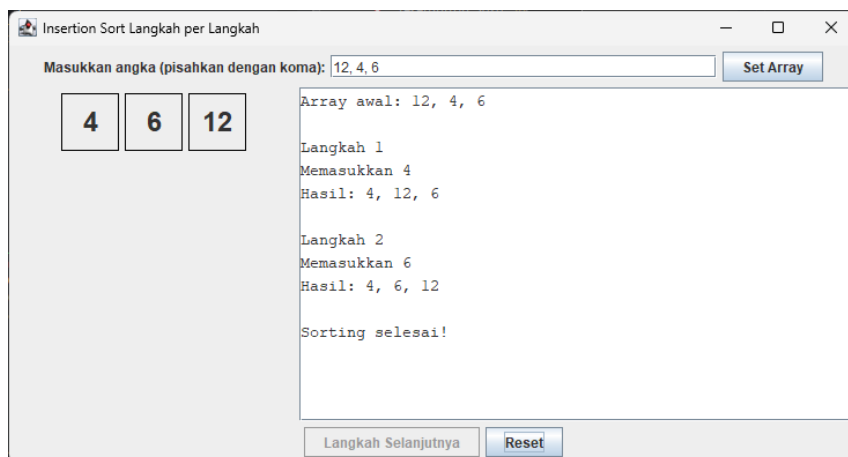
```

```

142 private void updateStatus_3026() {
143     for (int k_3026 = 0; k_3026 < array_3025.length; k_3026++) {
144         labelArray_3025[k_3026].setText(String.valueOf(array_3025[k_3026]));
145     }
146
147     private void reset_3026() {
148         inputField_3026.setText("");
149         panelArray_3026.removeAll();
150         panelArray_3026.revalidate();
151         panelArray_3026.repaint();
152         stepLog_3026.setText("");
153         stepButton_3026.setEnabled(true);
154         sorting_3025 = false;
155         i_3026 = 1;
156         stepCount_3026 = 1;
157         array_3025 = null;
158         labelArray_3025 = null;
159     }
160
161     private String arrayToString_3026(int[] arr_3025) {
162         StringBuilder sb_3026 = new StringBuilder();
163         for (int k_3026 = 0; k_3026 < arr_3025.length; k_3026++) {
164             sb_3026.append(arr_3025[k_3026]);
165             if (k_3026 < arr_3025.length - 1) {
166                 sb_3026.append(", ");
167             }
168         }
169         return sb_3026.toString();
170     }
171
172     public static void main(String[] args_3025) {
173         InsertionSortGUI_251133M204 = new InsertionSortGUI_251133M204();
174         InsertionSortGUI_251133M204.setVisible(true);
175     }
176 }

```

2.4.4 Hasil Output



2.4.5 Analisis

GUI sangat membantu dalam memahami proses internal Insertion Sort. Penggunaan event listener dan dynamic label membuat visualisasi menjadi interaktif dan edukatif.

2.6 Tugas Pekan 6

2.6.1 Analisis Kode Program

Program yang diimplementasikan pada praktikum pekan ini terdiri dari dua kelas utama, yaitu Mahasiswa_2511533026 dan SortingMahasiswaGUI_2511533026. Program ini merupakan aplikasi desktop berbasis GUI yang menggabungkan pengelolaan data mahasiswa dengan implementasi tiga algoritma sorting (Insertion Sort, Selection Sort, dan Bubble Sort).

Analisis Class Mahasiswa_2511533026 Kelas ini berfungsi sebagai data model atau objek mahasiswa. Menggunakan prinsip encapsulation dengan atribut private (nama_3026, nim_3026, dan prodi_3026). Dilengkapi dengan constructor, getter, setter, serta method toString() yang diformat rapi untuk keperluan tampilan tabel. Desain kelas ini sangat baik karena menjaga integritas data dan memudahkan manipulasi objek dalam ArrayList.

Analisis Class SortingMahasiswaGUI_2511533026 Kelas ini merupakan inti dari program yang mengextend JFrame. Fitur utama yang diimplementasikan meliputi:

- Manajemen Data: Menambah, menghapus, dan menampilkan data mahasiswa menggunakan ArrayList<Mahasiswa_2511533026> dan JTable dengan DefaultTableModel.
- Visualisasi Tabel: Data mahasiswa ditampilkan secara dinamis melalui method refreshTable_3026().
- Integrasi Algoritma Sorting: Menggunakan JComboBox untuk memilih algoritma (Insertion Sort, Selection Sort, atau Bubble Sort).
- Proses Sorting: Method mulaiSorting_3026() memanggil salah satu dari tiga algoritma sorting yang telah dimodifikasi untuk bekerja pada ArrayList dan mencatat setiap langkahnya ke JTextArea.
- Visualisasi Langkah demi Langkah: Setiap algoritma dilengkapi dengan Thread.sleep() untuk memberikan efek animasi sehingga pengguna dapat melihat proses pengurutan secara real-time.

Pergerakan Mahasiswa - NM 2511533026

Input Data Mahasiswa

Nama Mahasiswa:

NIM:

Program Studi:

Tambah Data

Hapus Data Tergilih

File Algorithm: Insertion Sort

Menu Sorting

Clear Semua

No	Nama	NIM	Program Studi
1	Ahmad	234	Informatika
2	Fah	123	Informatika
3	Phro	456	Informatika

Visualisasi Proses Sorting

--- INSERTION SORT ---

Langkah 1 : [Ahmad, Fah, Phro]

Langkah 2 : [Ahmad, Fah, Phro]

--- HASIL AKHIR SORTING ---

Ahmad	234	Informatika
Fah	123	Informatika
Phro	456	Informatika

Pergerakan Mahasiswa - NM 2511533026

Input Data Mahasiswa

Nama Mahasiswa:

NIM:

Program Studi:

Tambah Data

Hapus Data Tergilih

File Algorithm: Selection Sort

Menu Sorting

Clear Semua

No	Nama	NIM	Program Studi
1	Ahmad	234	Informatika
2	Fah	123	Informatika
3	Phro	456	Informatika

Visualisasi Proses Sorting

--- SELECTION SORT ---

Pass 1 : [Ahmad, Fah, Phro]

Pass 2 : [Ahmad, Fah, Phro]

--- HASIL AKHIR SORTING ---

Ahmad	234	Informatika
Fah	123	Informatika
Phro	456	Informatika

Pergerakan Mahasiswa - NM 2511533026

Input Data Mahasiswa

Nama Mahasiswa:

NIM:

Program Studi:

Tambah Data

Hapus Data Tergilih

File Algorithm: Bubble Sort

Menu Sorting

Clear Semua

No	Nama	NIM	Program Studi
1	Ahmad	234	Informatika
2	Fah	123	Informatika
3	Phro	456	Informatika

Visualisasi Proses Sorting

--- BUBBLE SORT ---

Pass 1 : [Ahmad, Fah, Phro]

Pass 2 : [Ahmad, Fah, Phro]

--- HASIL AKHIR SORTING ---

Ahmad	234	Informatika
Fah	123	Informatika
Phro	456	Informatika

BAB III

KESIMPULAN

Praktikum pekan ke-7 ini telah berhasil memberikan pemahaman yang mendalam mengenai tiga algoritma pengurutan dasar yaitu Insertion Sort, Selection Sort, dan Bubble Sort, serta penerapannya pada data mahasiswa dalam bentuk aplikasi Graphical User Interface (GUI). Melalui implementasi kode, kami dapat melihat secara langsung bagaimana masing-masing algoritma bekerja, mulai dari proses memasukkan data mahasiswa (nama, NIM, dan program studi), melakukan pengurutan berdasarkan nama, hingga menampilkan proses sorting secara visual langkah demi langkah. Penggunaan ArrayList beserta kelas Mahasiswa yang menerapkan konsep encapsulation (getter, setter, dan toString) membuat pengelolaan data menjadi lebih terstruktur dan mudah dikelola. Selain itu, integrasi Java Swing pada SortingMahasiswaGUI berhasil menghadirkan visualisasi interaktif yang sangat membantu dalam memahami alur kerja algoritma secara real-time.

Secara keseluruhan, praktikum ini tidak hanya memperkuat pemahaman teori algoritma sorting, tetapi juga mengasah kemampuan dalam menggabungkan struktur data, algoritma, dan pemrograman antarmuka pengguna. Ketiga algoritma yang diimplementasikan memiliki kelebihan dan kekurangan masing-masing, terutama dalam hal efisiensi dan adaptasi terhadap data. Pengalaman ini menjadi fondasi yang sangat berharga untuk mempelajari algoritma sorting yang lebih efisien seperti Quick Sort, Merge Sort, dan Heap Sort pada praktikum mendatang. Dengan demikian, tujuan praktikum untuk memahami konsep sorting beserta visualisasinya telah tercapai dengan baik.

DAFTAR PUSTAKA

1. Weiss, M. A. (2014). *Data Structures and Algorithm Analysis in Java*. Pearson.
2. Cormen, T. H., et al. (2009). *Introduction to Algorithms*. MIT Press.

3. Sedgewick, R., & Wayne, K. (2011). *Algorithms*. Addison-Wesley.
4. Materi Kuliah Struktur Data – Universitas Andalas.
5. Dokumentasi Java Swing – Oracle.[Oracle](https://docs.oracle.com/javase/7/docs/api/index.html).